

DOCUMENT 9: CODE REPOSITORY DETAILS

Project Name: PARIKSHAK-AI (परीक्षक-AI)

Challenge: Challenge 03 — AI-Driven Examination & On-Screen Marking Transformation

Track: Technical Track | **Team Name:** eMitra

Submission Field: 9. Code Repository URL *

1. Repository URL

```
https://github.com/nishchaydev/Mphack
```

The repository contains the working proof of concept (backend, frontend, tests, sample scripts), the 10 submission documents, and the run and demo guide (POC_GUIDE.md).

2. Run It in One Command

```
git clone https://github.com/nishchaydev/Mphack.git
cd Mphack
./run.sh                # Windows: run.bat
```

The first run creates a Python virtual environment and installs backend/requirements.txt (Python 3.11+). Then open:

- Examiner Workspace: <http://localhost:8000/>
- CoE Command Centre: <http://localhost:8000/coe>

Without an API key the AI panel uses bundled responses for the sample scripts and says so. For live AI, copy .env.example to .env and set GEMINI_API_KEY (and GEMINI_MODEL). Run the tests with `cd backend && ../../venv/bin/python -m pytest -q`.

3. Repository Structure

Mphack/	
├─ README.md	Project overview and architecture
├─ POC_GUIDE.md	Run guide, 5-minute demo script, real vs simulated
├─ run.sh / run.bat	One-command start
├─ .env.example	Configuration template (model, prices, thresholds)
├─ backend/	
│ └─ app/	
│ └─ main.py	FastAPI routes; serves both pages
│ └─ grading.py	Gemini multimodal call, prompt, JSON schema, cache, cost
│ └─ guards.py	Mark clamping, quote grounding, injection scan
│ └─ sentinel.py	Reading floor and tiers, seed drift, entropy
│ └─ workflow.py	Open → pre-read → submit → verify
│ └─ integrity.py	SHA-256, Merkle root, Ed25519 signer, hash-chained ledger
│ └─ dossier.py	Audit dossier PDF (Hindi text shaping)
│ └─ coe.py	CoE dashboard data
│ └─ schemas.py	Pydantic v2 data contracts
│ └─ config.py	Settings from environment / .env
│ └─ demo_data/	Questions, rubrics, script queue, saved AI responses
│ └─ tools/	
│ └─ make_samples.py	Generates the synthetic sample pages
│ └─ benchmark.py	Accuracy and cost against teacher marks
│ └─ tests/test_poc.py	15 automated tests
├─ frontend/	Examiner Workspace and CoE pages (HTML/CSS/JS, no build step)
├─ samples/	Synthetic handwritten answer pages
├─ docs/screenshots/	Screens used in Document 8
└─ submissions/	The 10 portal submission documents

4. Technology Used in the Proof of Concept

Layer	Technology
Backend	Python 3.11+, FastAPI, Pydantic v2
AI	Google Gemini multimodal API via the google-genai SDK, structured JSON output
Integrity	SHA-256, Merkle tree, Ed25519 signatures (cryptography), hash-chained ledger
Dossier PDF	fpdf2 with HarfBuzz shaping for Devanagari; Pillow for annotated pages
Frontend	HTML, CSS and JavaScript; canvas with Pointer Events for stylus pressure
Tests	pytest (15 tests)

The production architecture in Document 7 adds DocLayout-YOLO segmentation, PostgreSQL, Kafka, a React PWA and a self-hosted vision model in the State Data Centre. The proof of concept keeps the same interfaces so these can be added without changing the examiner workflow.